



UNIVERSAE

“MESERO APP”

PROYECTO INTERMODULAR DE (LÍNEA 2)

**CICLO FORMATIVO DE GRADO SUPERIOR DE DESARROLLO DE
APLICACIONES MULTIPLATAFORMA**

ROBERTO DE FRUTOS JIMÉNEZ

ALFONSO GARCÍA

Curso 2025-2026

Código del proyecto: [GITHUB](#)



ÍNDICE

1. Resumen.....	3
2. Abstract	4
3. Introducción	5
4. Descripción y Objetivos	7
5. Contenidos	8
6. Metodología y planificación (2-4 páginas)	14
7. Desarrollo (4-8 páginas).....	16
8. Conclusiones y proyección a futuro (1-2 páginas)	24
9. Bibliografía	25
10. Anexos	27



1. Resumen

MESERO APP, una aplicación móvil desarrollada en Java para dispositivos con sistema operativo Android, esta se basa en hacer más fácil la gestión diaria de negocios de hostelería como bares y restaurantes, especialmente se encarga de organizar las mesas, comandas, stock y facturación de pequeños y medianos negocios. La idea surge del problema real de personas conocidas que se dedican a trabajar en este sector para así ayudar a optimizar el tiempo en lo máximo posible, evitar errores humanos y evitar el cansancio de camareros. Otra de las ventajas de esta digitalización de negocios es evitar el uso de hojas para tomar nota de las comandas y permitir enviar la factura final así consiguiendo una mayor sostenibilidad con una menor contaminación. La arquitectura seleccionada para realizar este proyecto fue MVVM (Model-View-ViewModel), un modelo que permite organizar la aplicación en niveles funcionales facilitando la programación y mantenimiento a largo plazo (Torunoğlu, 2024) encargado de gestionar la lógica de la aplicación además de las capas con conexión a la base de datos. El uso de Figma permitió la creación de un prototipo de la interfaz (Figma, n.d.) para así realizarlo de manera sencilla, el uso de LiveData permitió la actualización en tiempo real en las listas utilizadas (Android Developers, n.d.-a). Además, el uso de fragmentos para reutilizar vistas permite un consumo menor de memoria del dispositivo, verificaciones en 2 pasos en el proceso de login y registro y SMTP para realizar envíos de correos electrónicos (GeeksforGeeks, 2025). El desarrollo en Android se justifica por su amplia cuota de mercado a nivel mundial (StatCounter Global Stats, 2026). Finalmente, se siguió una metodología ágil basada en ciclos iterativos (Asana, 2025). A lo largo del proyecto se llevaron a cabo pruebas de caja blanca para la comprobación de funcionamiento lógico y funcional de la aplicación. Como resultado se ha conseguido una herramienta funcional, simple para el usuario y gratuita que permita la gestión de las principales necesidades en el mundo de la hostelería.

Palabras clave: Android, automatización, hostelería, java, aplicación móvil



2. Abstract

MESERO APP is a mobile application developed in Java for devices running the Android operating system. It is designed to simplify the daily management of hospitality businesses such as bars and restaurants, specifically by organizing tables, orders, stock, and billing for small and medium-sized establishments. The idea emerged from a real problem experienced by professionals working in this sector, with the aim of optimizing time, reducing human errors, and minimizing waiter fatigue. Another advantage of this business digitalization is the reduction of paper usage for taking orders and issuing final invoices, thereby promoting greater sustainability and reducing environmental impact. The architecture implemented in this project is MVVM (Model-View-ViewModel), a pattern that separates responsibilities into different layers, resulting in more maintainable and testable code (Torunoğlu, 2024), while managing the application logic and database connection layers. Figma was used to design the user interface prototype (Figma, n.d.), facilitating a structured development process. LiveData was implemented to enable real-time updates of the displayed lists (Android Developers, n.d.-a). Additionally, the use of fragments allows view reuse, reducing device memory consumption. Two-step verification was incorporated in the login and registration processes, and SMTP was used for sending email notifications (GeeksforGeeks, 2025). The choice of Android development is justified by its wide global market share (StatCounter Global Stats, 2026). Finally, an agile methodology based on iterative cycles was followed during development (Asana, 2025). White box testing was conducted to verify the logical and functional performance of the application. As a result, a functional, user-friendly, and free tool was developed to address the main management needs within the hospitality sector.

Keywords: Android, automation, hospitality, Java, mobile application.

3. Introducción

Figura 1

Logo de MESERO APP



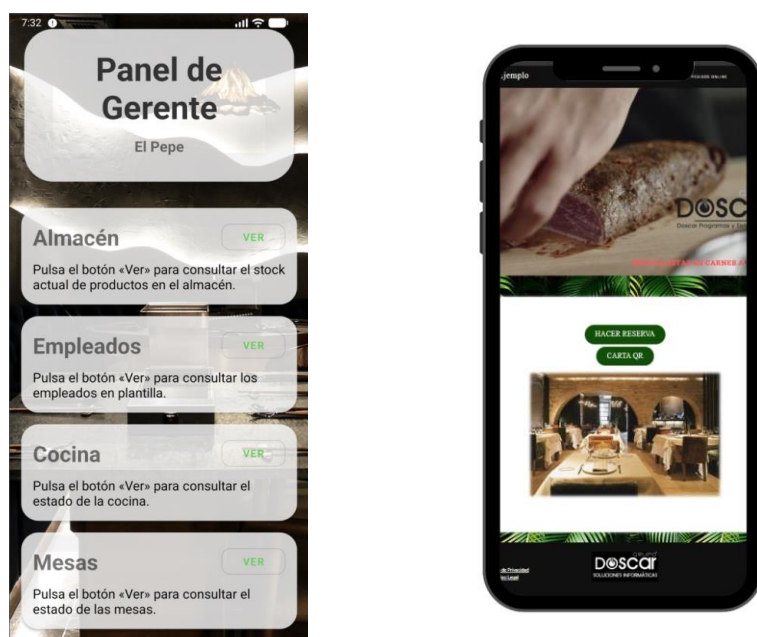
Mesero App es una aplicación móvil creada como la (v2) de un proyecto personal desarrollado por Roberto de Frutos Jiménez 2025. Esta esta realizada como una web y la usabilidad de esta quedaba reducida por lo que se decidió crear esta solución de automatización y digitalización en una aplicación móvil para pequeños y medianos negocios dedicados a la hostelería, la idea de esta fue ayudar a conocidos pertenecientes a este sector a realizar las tareas diarias de manera más sencilla y eficiente.

Las tecnologías utilizadas para realizar esta aplicación son Figma (Figma, n.d.) para el prototipado de interfaz, GitHub Copilot (GitHub, 2024). para auto-rrellenado de documentación y ayuda en la corrección de errores, Java para la creación de la lógica y backend de la aplicación, SQLite y SharedPreferences para el guardado de la sesión de usuarios, SMTP para envío de correos (Postel, 1981), y Android studio como entorno de desarrollo (Android Developers, n.d.).

En el mercado actual de aplicaciones móviles existen alternativas como el programa “DOSCAR restaurante”, el cual; es líder en el sector, permite la gestión de bares y restaurantes de manera profesional desde dispositivos TPV y tablets, este programa en comparación con Mesero presta estas automatizaciones con un precio de 164,95€ – 613,95€ + IMPUESTOS en comparación con el programa desarrollado de carácter gratuito.

Figura 2

Interfaces de Mesero APP y Doscar





4. Descripción y Objetivos

La aplicación se encuentra actualmente en una versión beta, tras las pruebas de caja blanca realizadas se está actualmente intentando implementar en un entorno de hostelería real para realizar uso de esta y sus automatizaciones. (GeeksforGeeks, 2025).

La aplicación esta realizada con una interfaz lo más sencilla e intuitiva posible para que empleados en hosteleria de todas las edades puedan hacer uso de esta sin necesidad de gastar dinero para mejorar la atención al cliente y gestión del negocio. (Norman, 2013).

Se implementa una funcionalidad de verificación 2FA para asegurar los riesgos de acceso no autorizado (National Institute of Standards and Technology [NIST], 2017), la separación de responsabilidades para los distintos roles que se encuentran en los restaurantes, la gestión de mesas (ver si la mesa esta libre, tiene una reserva o está ocupada), añadiendo comandas a las mesas ocupadas y restando automáticamente el stock de los productos que los clientes gastan, el uso de notificaciones para avisar a la cocina de la llegada de una nueva comanda a través de sonido y al gerente para avisar del poco stock restante en productos además del envío de facturas a los clientes a través de correo electrónico.

En conjunto, estos objetivos permiten estructurar el desarrollo por fases y asegurar que la aplicación responda de forma directa a la necesidad de digitalización y mejora organizativa detectada en el sector.

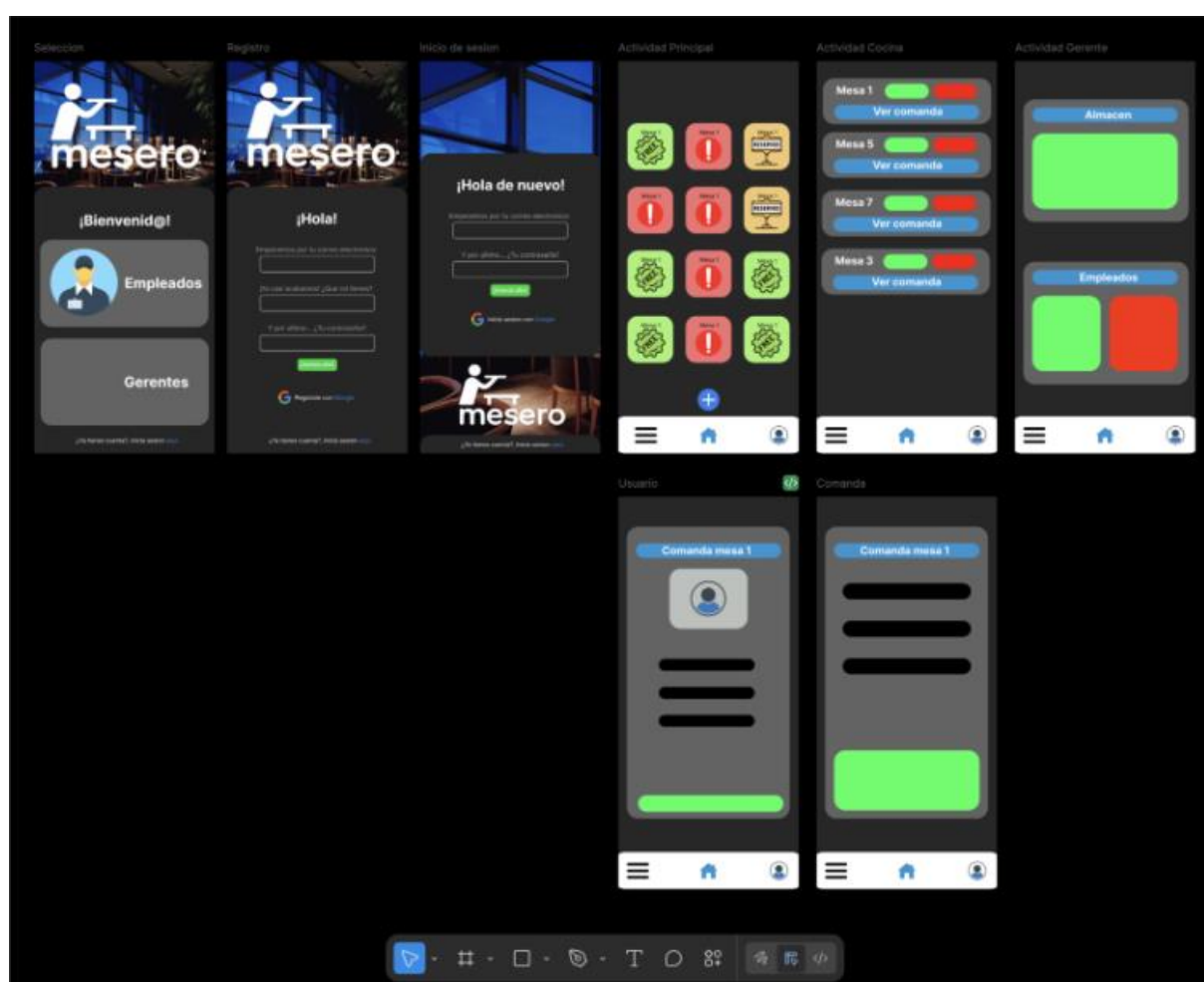
5. Contenidos

Para anticipar visualmente la interfaz de usuario con los principales requerimientos de esta se utiliza la aplicación web gratuita de Figma (Figma, n.d.), permitiendo así crear las vistas y actividades necesarias para cada uno de los apartados y roles de usuarios que utilicen la aplicación. La creación de este prototipo permite a los programadores una idea más específica del diseño necesario para que el programa cumpla los requisitos necesarios.

Figura

4

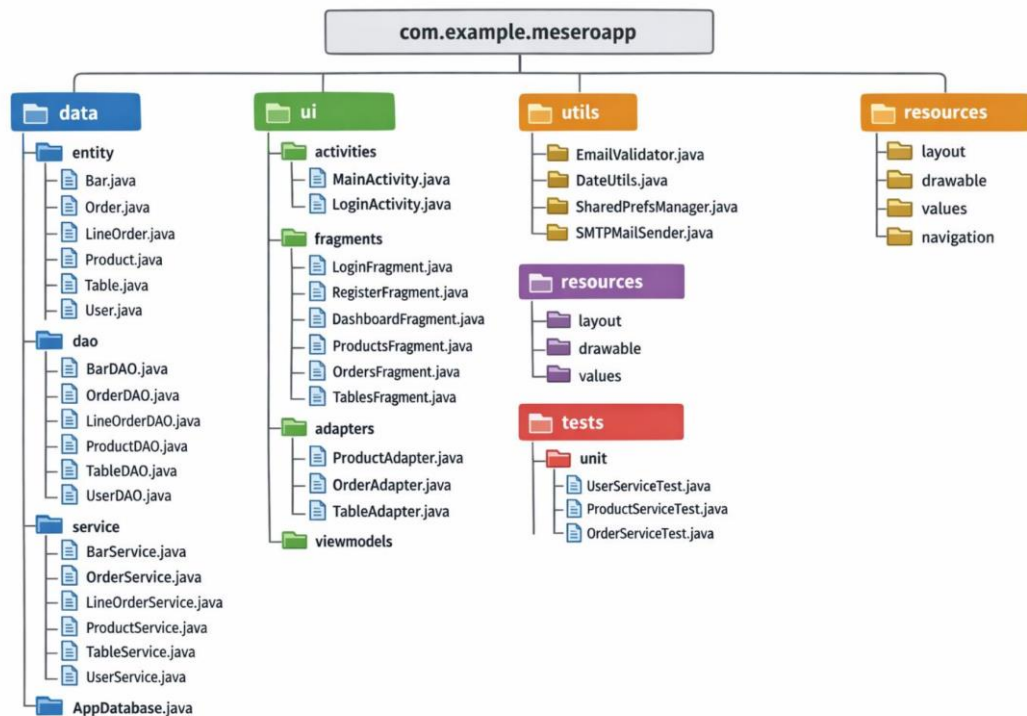
Captura de pantalla del prototipo creado en Figma



El proyecto se creó con una arquitectura MVVM (Torunoğlu, 2024) por lo que este se divide desde la carpeta raíz en varias carpetas “hijo” para mantener la separación de responsabilidades y organización del proyecto así permitiendo la escalabilidad y el código limpio en este. Como se percibe en la siguiente imagen; las principales carpetas son:

Figura 5

Imagen esquemática sobre la división del código en Mesero



Data:

- Entity: Son las principales entidades y objetos necesarios para guardar estos en la base de datos.
- DAO: Son clases para recibir y enviar información a la UI de manera segura y organizada, similar a el uso en API's, esta recibe los parámetros necesarios para realizar la conversión y enviar a la base de datos lo necesario.
- Service: En este apartado encontramos la lógica de la aplicación, desde la creación de un objeto al tocar cierto botón a él filtrado de objetos.
- AppDatabase: Son los ficheros necesarios para conseguir la persistencia de objetos en la base de datos, esto realiza qué; al guardarlo en la base de datos, la aplicación no pierde los datos generados al cerrar este proceso. (SQLite Consortium, n.d.).



UI:

En este apartado encontramos lo necesario para mostrar toda la información al usuario, en esta encontramos tan solo 2 actividades que pueden ser usadas para todas las funcionalidades de la aplicación, esto se consigue reutilizando la vistas de estas con la siguiente carpeta, los fragmentos (Android Developers, n.d.); esto es la vista personalizada que cada usuario puede tener dependiendo del rol que ejerza dentro del restaurante, este se decide en el registro de usuarios o el gerente del restaurante lo puede cambiar desde su menú.

Utils y resources:

En estos dos apartados se guardan algunas de las funcionalidades más tarde utilizadas por service, al separar esta lógica puede ser reutilizada para los diferentes fragmentos de la aplicación. En la carpeta resources se guardan todas las imágenes necesarias para el icono y la interfaz de usuario pudiendo utilizar estas a través de ids.

Las principales funcionalidades implementadas en la aplicación son:

- Registro de usuarios y bares de manera segura con autenticación de doble factor (2FA) (National Institute of Standards and Technology [NIST], 2017)
- Interfaz sencilla e intuitiva para la creación de mesas y stock en el negocio.
- Envío de notificaciones al recibir nuevos pedidos en cocina.
- Envío de correo electrónico con la factura a los clientes. (Postel, 1981).
- Gestión de empleados por parte del gerente en su menú principal, permitiendo el cambio de rol, nombre o email de estos
- Automatización en el inventario de stock al restar las cantidades gastadas en cada orden (Laudon & Laudon, 2020).



En la siguiente imagen se puede ver de manera estructurada como se guardan las diferentes entidades necesarias para realizar estas funcionalidades en la base de datos, permitiendo guardar y relacionar estas desde la subcarpeta de entidades en la que se declaran las relaciones y claves primarias y foráneas.

Figura 6 y 7

Capturas de pantallas representando como relacionar entidades y guardar las "PK"

```

1 package data.entity;
2
3 > import ...
7
8 @Entity(indices = {@Index(value = "email", unique = true)})
9 public class Bar {
10     @PrimaryKey(autoGenerate = true)
11     public int id;
12     public String barName;
13     public String email;
14
15     public Bar() {}

```

```

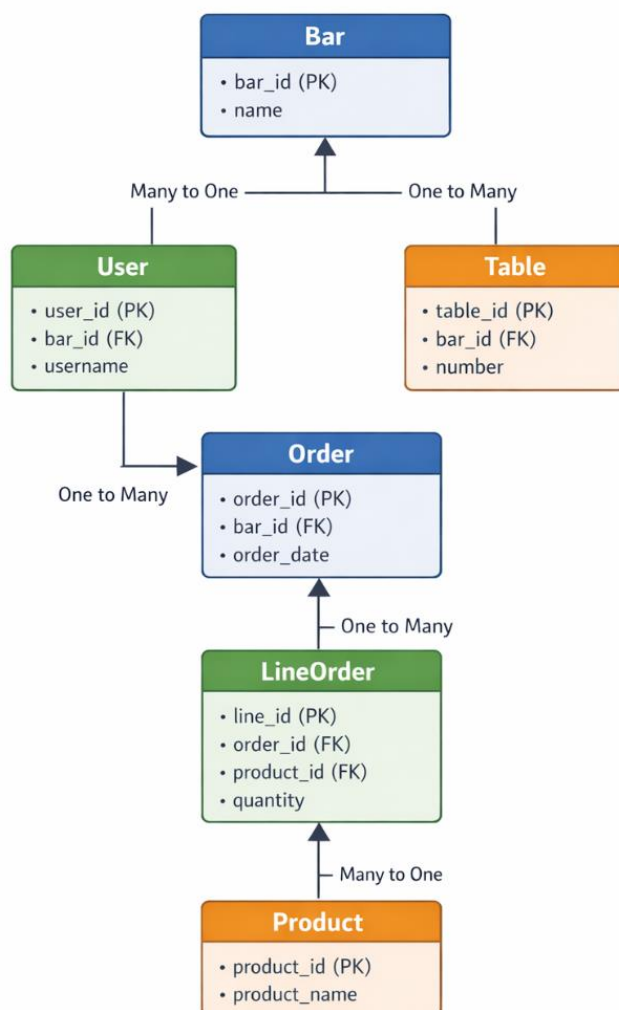
1 package data.entity;
2
3 > import ...
7
8 public class BarWithOrder {
9     @Embedded
10     public Bar bar;
11
12     @Relation(
13         parentColumn = "id",
14         entityColumn = "barId"
15     )
16     public List<Order> orders;
17 }
18

```

Relacionando con lo anterior, podemos realizar así de manera automática una gestión de la base de datos de manera sencilla consiguiendo un diagrama de clases como el siguiente:

Figura 8

Imagen representando el diagrama de clases de la aplicación





Por último, en este punto consiguiendo con esta persistencia y funcionalidades los siguientes casos de uso:

Casos de uso con usuario gerente:

Crear un nuevo producto:

- Accede a el panel de gestión
- Introduce nombre, precio y stock
- El sistema guarda el producto

Crear una mesa:

- Pulsa el botón + en vista camarero.
- Introduce el número y capacidad.
- El sistema guarda el producto

Casos de uso con usuario camarero:

Crear orden:

- El usuario inicia sesión.
- Selecciona la mesa de la comanda.
- Añade productos a la orden y se resta stock
- Confirma pedido
- La comanda se envía a cocina

Añadir productos a la orden:

- El usuario selecciona la mesa con orden abierta.
- Añade los productos a la comanda
- El programa recalcula el total

Cerrar orden:

- El usuario selecciona la mesa con orden abierta.
- Envía la factura por correo si es necesario.
- La mesa aparece como libre



6. Metodología y planificación

En cuanto a la planificación del proyecto se lleva a cabo tras consultas con GitHub Copilot asistido por modelos de lenguaje avanzados (GitHub, 2024). tras algunos cambios realizados en esta se lleva a cabo la siguiente planificación con fecha límite el 8 de marzo de 2026 para así se pueda realizar la revisión académica por parte del tutor.

Durante este periodo se abordará la fase final del desarrollo de la aplicación, centrada en la consolidación de funcionalidades, optimización, pruebas y entrega. (Asana, 2025).

En la primera semana (16–22 de febrero) se implementará la vista de cocina para gestionar comandas y estados de los pedidos, incluyendo notificaciones internas y actualización automática del estado global. También se optimizarán las consultas de Room y el uso de asincronía para garantizar un rendimiento adecuado.

En la segunda semana (23 de febrero–1 de marzo) se desarrollará el sistema de exportación e importación de datos para copias de seguridad. Se realizará el diseño final de la interfaz, unificando estilos y mejorando la experiencia de usuario. Además, se revisarán todas las validaciones (email único, control de stock, reservas y manejo de errores) (Android Developers, n.d.).

En la última semana (2–8 de marzo) se llevarán a cabo pruebas completas en distintos dispositivos para verificar persistencia y funcionamiento del flujo general de la aplicación. Finalmente, se elaborará la documentación técnica, el README y se generará el APK firmado para la entrega.

Objetivo: entregar una aplicación funcional, optimizada y correctamente documentada para su evaluación académica.



Como se puede observar la planificación se estructuró en periodos cortos de trabajo, organizando tareas específicas por semanas y revisando progresivamente el avance del proyecto, permitiendo detectar errores rápidamente. Así mismo esta metodología ágil permite una mayor flexibilidad adaptándose al cambio constante del código en este software, consiguiendo así una evolución progresiva de la aplicación hasta alcanzar una versión final estable y funcional.

Para la creación de este proyecto fue necesario conocer lo máximo posible en el lenguaje de programación Java para la lógica de negocio y XML junto con el sistema de vistas de Android para el diseño de interfaces. Además del uso del entorno de desarrollo Android Studio con la integración de GitHub Copilot para preguntar dudas y creación de documentación y comentarios dentro del código.

Tabla 1

Gestión de tiempo y planificación, generada con GitHub Copilot Caude

Semana	Tareas	Horas Estimadas	Dificultad
Semana 1 16–22 de febrero	• Vista de Cocina y notificaciones • Optimización Room y asincronía	20 horas	Alta ●●●●
Semana 2 23 de febrero–1 de marzo	• Exportación/Importación de datos • Diseño final de UI y validaciones	18 horas	Media ●●●●
Semana 3 2–8 de marzo	• Pruebas finales en dispositivos • Documentación y entrega final	22 horas	Media-Alta ●●●●



7. Desarrollo

En primer lugar, en este punto presentare las elecciones tomadas para realizar este proyecto pensando siempre en la mayor seguridad, simpleza y compatibilidad para los usuarios. (StatCounter Global Stats, 2026).

Se seleccionó la API 21 (Android 5.0) con el objetivo de alcanzar una alta compatibilidad con dispositivos Android aún en uso para que la mayoría de los teléfonos Android del mercado puedan utilizar esta aplicación sin restricciones. El lenguaje utilizado es Java para realizar la lógica de negocio y Java FX para realizar las interfaces de usuario.

Para el envío de verificaciones mediante autenticación de doble factor (2FA) y el envío de facturas electrónicas, se implementó el protocolo SMTP. Esta elección permite mantener independencia respecto a proveedores externos y facilita la portabilidad, escalabilidad e integración con el backend (Postel, 1981). Desde el punto de vista de seguridad, la incorporación de 2FA sigue las recomendaciones internacionales en materia de identidad digital (National Institute of Standards and Technology [NIST], 2017).

Para mantener una gran eficiencia y poco uso de recursos se opta por usar los fragmentos dentro de actividades permitiendo así mostrar varios fragmentos en una única actividad.

Para proceder con la creación de la aplicación se opta por un diseño diagonal, es decir hacer las interfaces de usuario y la lógica a la vez permitiendo así realizar las pruebas unitarias oportunas (Asana, 2025).

En este punto se presenta el desarrollo realizado, en primer lugar, se crearán todas las subcarpetas necesarias para realizar las clases pertinentes dentro de estas:

Para comenzar con el desarrollo se crearán dentro de la carpeta data todos los objetos necesarios para realizar la funcionalidad conocidos como model o entity y sus respectivas relaciones entre sí para la persistencia en base de datos, además de estos se crearán los DTO's necesarios para presentar la información y guardar esta desde



la interfaz de usuario, en las siguientes imágenes se encontrarán ejemplos de lo mencionado.

Figura 9

Imagen de los elementos creados para el desarrollo

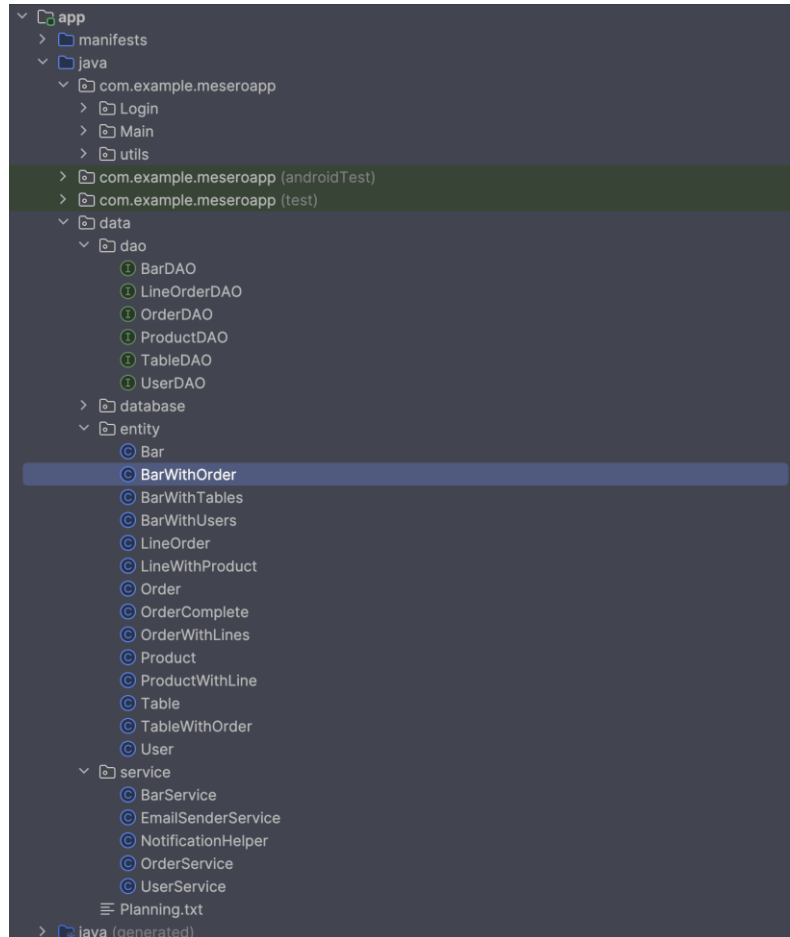


Figura 10 y 11

Imágenes de una clase entity y su relación con otra clase

```

1 package data.entity;
2
3 > import ...
7
8 @Entity(indices = {@Index(value = "email", unique = true)})
9 public class Bar {
10     @PrimaryKey(autoGenerate = true)
11     public int id;
12     public String barName;
13     public String email;
14
15     public Bar() {}
16
17     @Ignore // Para que room utilice el otro room
18     public Bar(int id, String barName, String email) {
19         this.id = id;
20         this.barName = barName;
21         this.email = email;
22     }
23
24     public int getId() { return id; }
27     public void setId(int id) { this.id = id; }
30     public String getBarName() { return barName; }
33     public void setBarName(String barName) { this.barName = barName; }
36     public String getEmail() { return email; }
39     public void setEmail(String email) { this.email = email; }
42

```

```

1 package data.entity;
2
3 > import ...
7
8 public class BarWithOrder {
9     @Embedded
10     public Bar bar;
11
12     @Relation(
13         parentColumn = "id",
14         entityColumn = "barId"
15     )
16     public List<Order> orders;
17 }

```

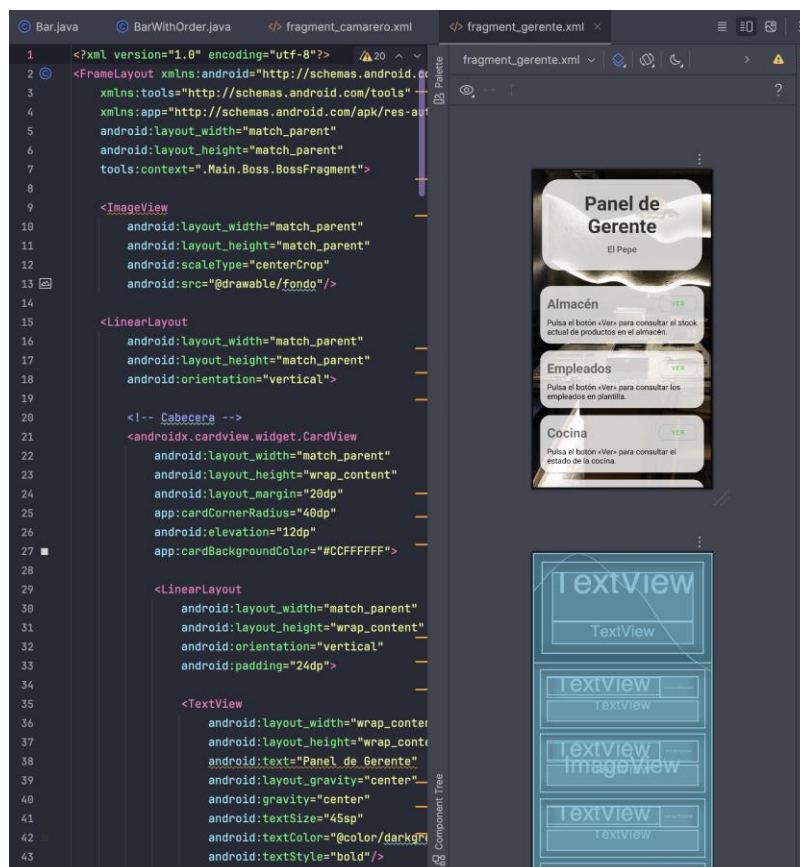
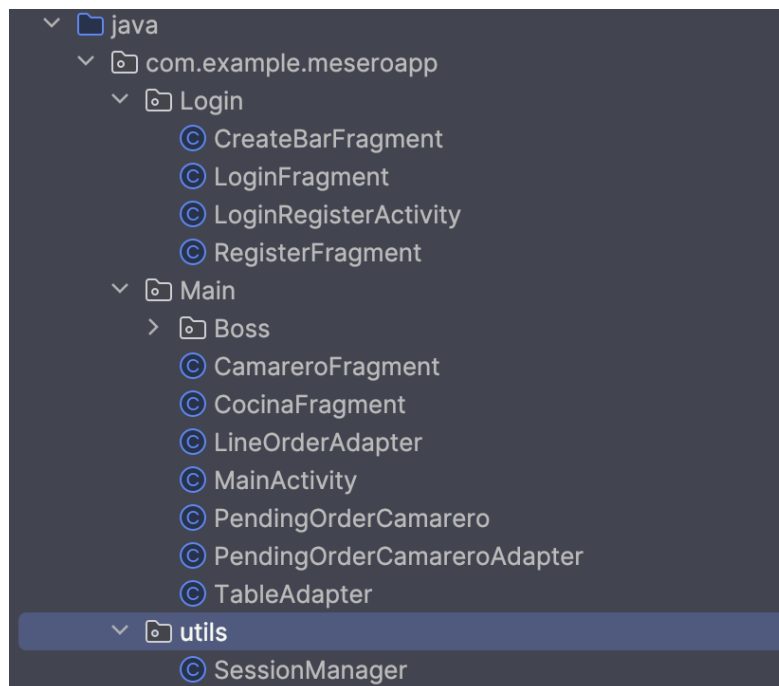
Una vez finalizada la creación de estos elementos se crean las interfaces de usuario necesarias, tanto la logica de estas y el diseño son creadas en esta fase

Figuras

11

y

12



Una de las decisiones más importante tomada para el correcto funcionamiento de la interfaz es el uso de fragmentos reutilizando así las actividades creadas, en este punto

se crea la lógica de inflado de estas actividades con los fragmentos necesarios a mostrar dependiendo del rol de cada empleado.

En el siguiente paso se opta por la creación de los servicios, programando así las principales utilidades de la aplicación separando estas en ficheros dependiendo del fragmento en el que se necesite la funcionalidad, el uso de servicios separando responsabilidades se crea para así poder reutilizar funcionalidades en diferentes ficheros de la aplicación.

```

1 package data.service;
2
3
4 import data.dao.BarDAO;
5 import data.entity.Bar;
6
7 public class BarService {
8
9     12 usages
10     private final BarDAO barDao;
11
12     1 usage
13     public BarService(BarDAO barDao) { this.barDao = barDao; }
14
15     1 usage
16     public long createBar(Bar bar) {
17         Bar existBar = barDao.getByBarName(bar.getBarName());
18
19         if (existBar != null) {
20             return -1; // Ya existe
21         }
22
23         return barDao.insert(bar);
24     }
25
26     no usages
27     public boolean deleteBar(int id) {
28         Bar bar = barDao.getId(id);
29
30         if (bar != null) {
31             barDao.delete(bar);
32             System.out.println("Bar '" + bar.getBarName() + "' borrado correctamente");
33             return true;
34         }
35
36         System.out.println("Bar no encontrado");
37         return false;
38     }
39
40     1 usage
41     public boolean existsBarByEmail(String email) {
42         Bar bar = barDao.getByEmail(email);
43         return bar != null;
44     }
45 }

```

Como se ve en la imagen en los servicios se implementa el propio objeto hacia el que este está guiado, realizando el uso de DTOS se realizan de manera segura las gestiones necesarias con la base de datos, estos servicios se usaran posteriormente en cada fragmento añadiendo así la funcionalidad deseada.



Para continuar con el desarrollo es necesaria la persistencia y realizar así las pruebas unitarias pertinentes para ver si realmente funcionan las utilidades implementadas, para ello se hace uso de los ficheros Sesión; guardando en este la información del usuario que inicie sesión y el fichero AppDataBase, el cual; es la conexión a la base de datos.

Para el correcto funcionamiento del guardado de sesión se añaden unos parámetros como el rol, nombre de usuario o email en el fichero permitiendo guardar así de manera local el inicio de sesión para permitir al usuario acceder a la app sin iniciar sesión teniendo que iniciarse antes una solo vez.



```

1 package com.example.meseroapp.utils;
2
3 > import ...
4
5 28 usages
6 public class SessionManager {
7
8     1 usage
9     private static final String PREF_NAME = "MeseroAppPrefs";
10    3 usages
11    private static final String KEY_BAR_ID = "barId";
12    2 usages
13    private static final String KEY_USER_ID = "userId";
14    2 usages
15    private static final String KEY_IS_LOGGED = "isLoggedIn";
16    3 usages
17    private static final String KEY_USER_ROLE = "userRole";
18
19    3 usages
20    private static SessionManager instance;
21    6 usages
22    private final SharedPreferences prefs;
23    12 usages
24    💡 private final SharedPreferences.Editor editor;|
25
26    1 usage
27    @ private SessionManager(Context context) {
28        |     prefs = context.getApplicationContext()
29        |         .getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE);
30        |     editor = prefs.edit();
31    }
32
33

```

Tabla 2 [ANEXO DE PRUEBAS.](#)

Resumen de pruebas y validaciones realizadas en MESERO APP.

	Módulo	Tipo de Error	Solución	Resultado
1	Gestión Bar	Validación de código incorrecto	Control de validación en dialogo	Creación bloqueada correctamente
2	Registro	Enviar token antes de validar email	Reordenación de validaciones	Registro bloqueado correctamente
3	Login	Sesión no persistente	Implemento ServiceManager	Sesión mantenida
4	Mesas / Productos	Ajuste visual y validaciones	Mejora de layout y control de datos	Funciona correctamente
5	Comandas	FragmentContender no existe	Corrección de ID en replace()	Sin cierres inesperados
6	Roles	Rol no almacenado	Añadimos el rol en Session Manager	Navegación correcta
7	Código bar	ID no actualizado en la vista	Corrección en la obtención de datos	Vista correcta
8	Cocina	Error de casteo	Corrección en tipo de componente	Fragmento estable
9	Almacén	Falta de validaciones	Validación automática en los campos	Gestión correcta
10	Pedidos	Flujo completo funcional	Validación integral	Sin errores, proceso correcto
11	Base de datos	Acceso en hilo principal	Accedo a la base de datos en hilo secundario	Error eliminado
12	Facturación	Error en el cálculo de total	Corrección en consulta DAO	Total, correcto



8. Conclusiones y proyección a futuro

Conclusiones:

Con el desarrollo de MESERO APP se aprendieron elementos clave para futuros proyectos. Se reforzó la importancia de la planificación con tareas específicas di (iteraciones cortas con revisiones constantes). La arquitectura por capas (MVVM) permitió separar responsabilidades y facilitar el mantenimiento del código. El uso de pruebas de caja blanca garantizó que la estructura interna del software cumpliera con los requisitos y mejoró la calidad del producto. También se adquirió experiencia práctica en el manejo de componentes Android (Room, LiveData, Fragments) y herramientas de diseño (Figma). Las dificultades principales incluyeron configurar la comunicación SMTP y manejar correctamente los hilos de UI; se solucionaron creando un servicio dedicado (EmailSenderService) y utilizando LiveData para la sincronización automática. Con respecto a los servicios y utilidades más demandadas en el mercado actual se realizan verificaciones 2FA tanto en el registro del bar como en el registro de usuarios en ese bar.

Proyección

futura:

En versiones posteriores se considera agregar sincronización en la nube (por ejemplo, migrar la persistencia a Firebase) para multi-dispositivo. También se podría integrar una pasarela de pago dentro de la aplicación. Una versión multiplataforma (p.ej., con Flutter o Kotlin Multiplatform) permitiría llegar a iOS. Para redes locales, se planea soporte multiusuario concurrente y desarrollar un panel web administrativo que consuma la misma base de datos. Como nuevas funcionalidades se puede asignar un contador de horas en fichaje a través de un dispositivo como una tarjeta y un ordenador o TPV que se encuentre en la localización asignada del restaurante o bar.



9. Bibliografía

Android Developers. (n.d.-a). *LiveData overview*. Google.
<https://developer.android.com>

Android Developers. (n.d.-b). *Save simple data with SharedPreferences*. Google.
<https://developer.android.com>

Android Developers. (n.d.-c). *Fragments*. Google. <https://developer.android.com>

Android Developers. (n.d.-d). *Room Persistence Library*. Google.
<https://developer.android.com>

Asana. (2025). *What is Agile methodology? A beginner's guide*. <https://asana.com>

DOSCAR. (n.d.). *Software TPV para restaurantes*. <https://www.doscar.com>

Figma. (n.d.). *Free prototyping tool: Build interactive prototype designs*.
<https://www.figma.com>

GeeksforGeeks. (2025). *White box testing – Software engineering*.
<https://www.geeksforgeeks.org>

GitHub. (2024). *What is version control?* <https://github.com>



Laudon, K. C., & Laudon, J. P. (2020). *Management information systems: Managing the digital firm* (16th ed.). Pearson.

National Institute of Standards and Technology. (2017). *Digital identity guidelines* (NIST Special Publication 800-63). U.S. Department of Commerce.

Postel, J. (1981). *Simple Mail Transfer Protocol (SMTP)* (RFC 821). Internet Engineering Task Force.

SQLite Consortium. (n.d.). *About SQLite*. <https://www.sqlite.org>

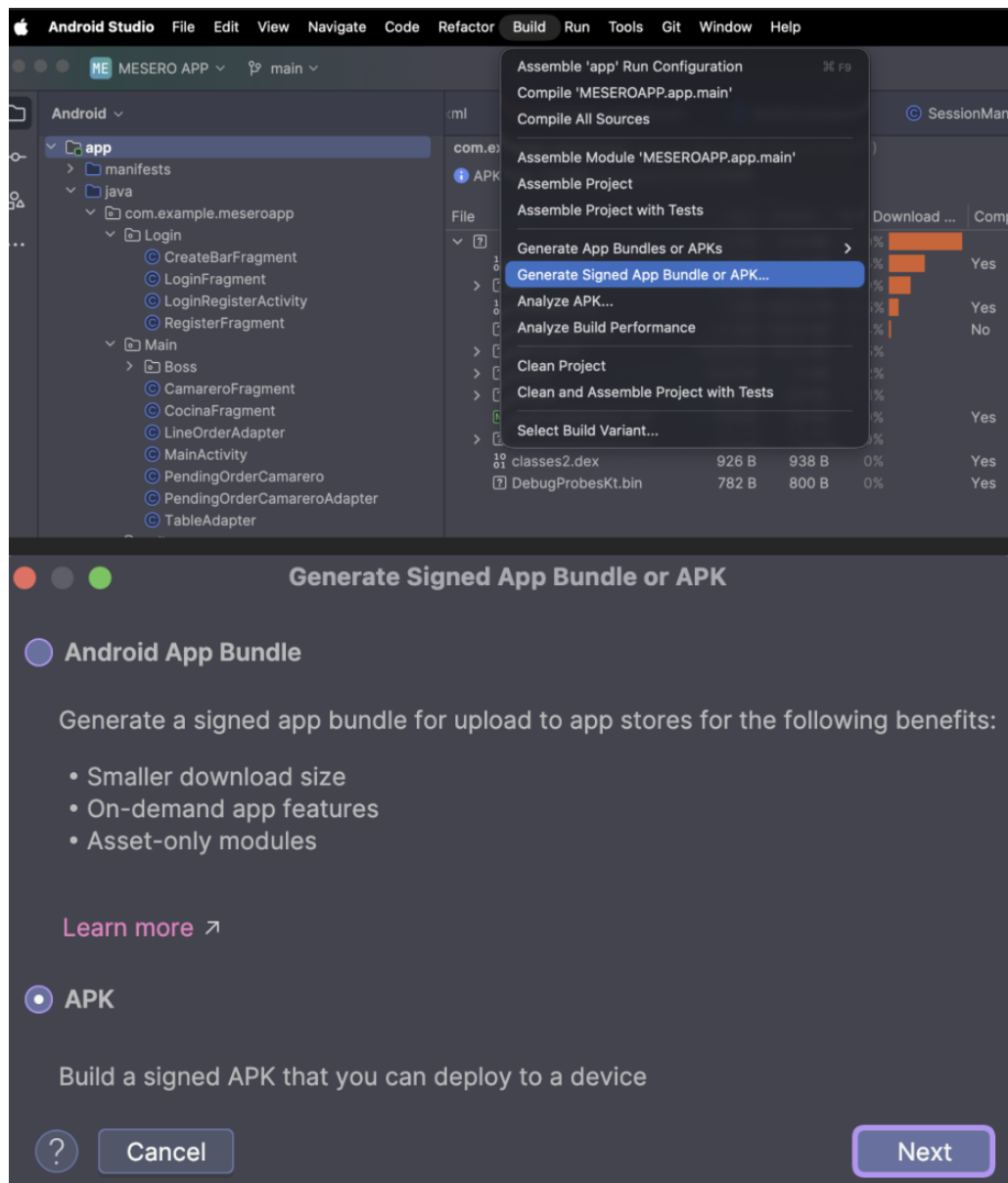
StatCounter Global Stats. (2026). *Mobile operating system market share worldwide*. <https://gs.statcounter.com>

Torunoğlu, Z. (2024). *MVVM on Android*. Medium. <https://medium.com>

10. Anexos

Despliegue e instalación:

Para desplegar la aplicación necesitamos seleccionar la opción build dentro de Android Studio en la parte superior, aquí seleccionamos la parte remarcada en azul y seleccionamos APK:

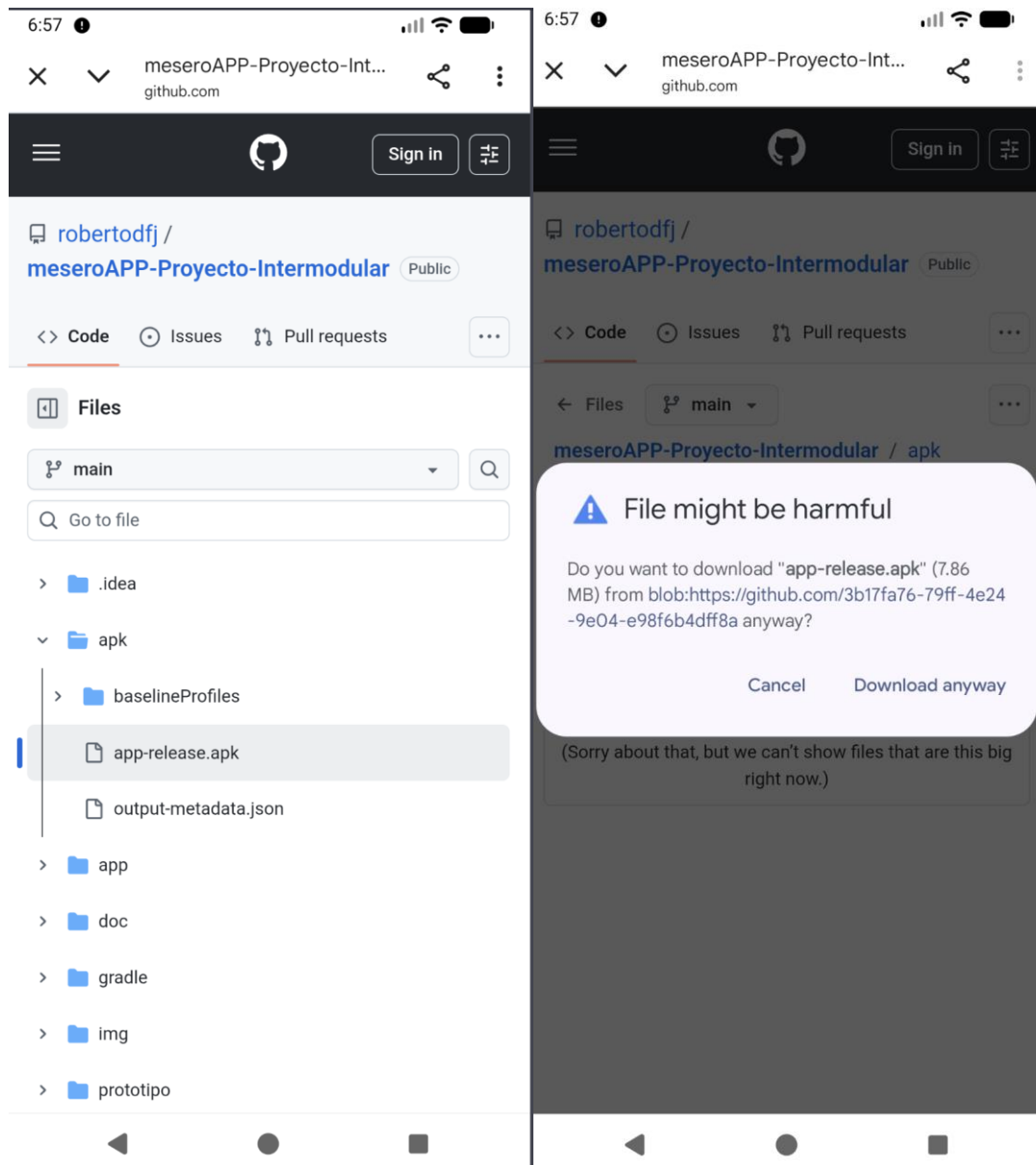


Añadimos la opción de crear una nueva “key” y añadimos los parametros necesarios que aparecen en la imagen:

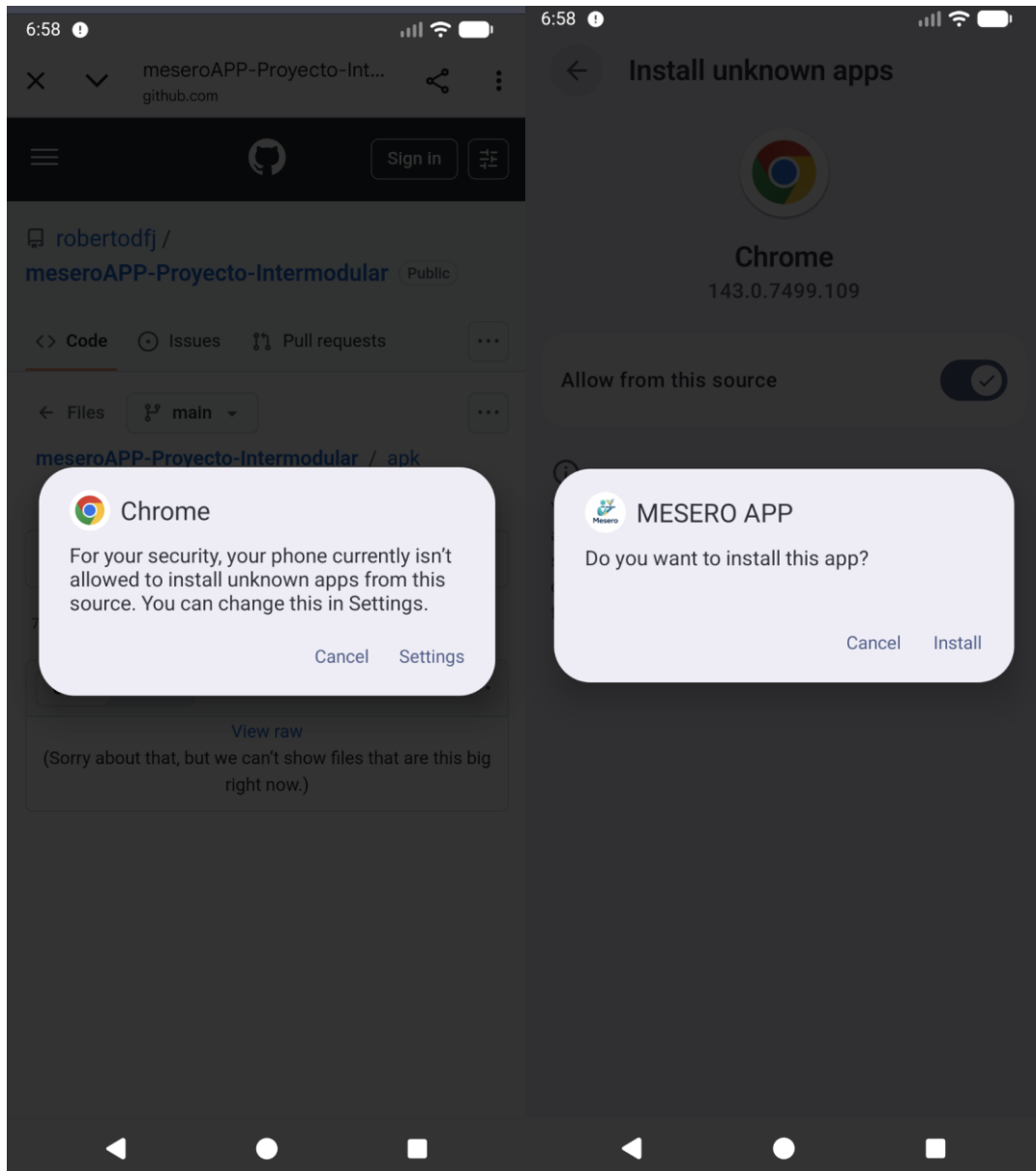
Seleccionamos la opción de release y pulsamos el botón, la APK se genera y se puede instalar



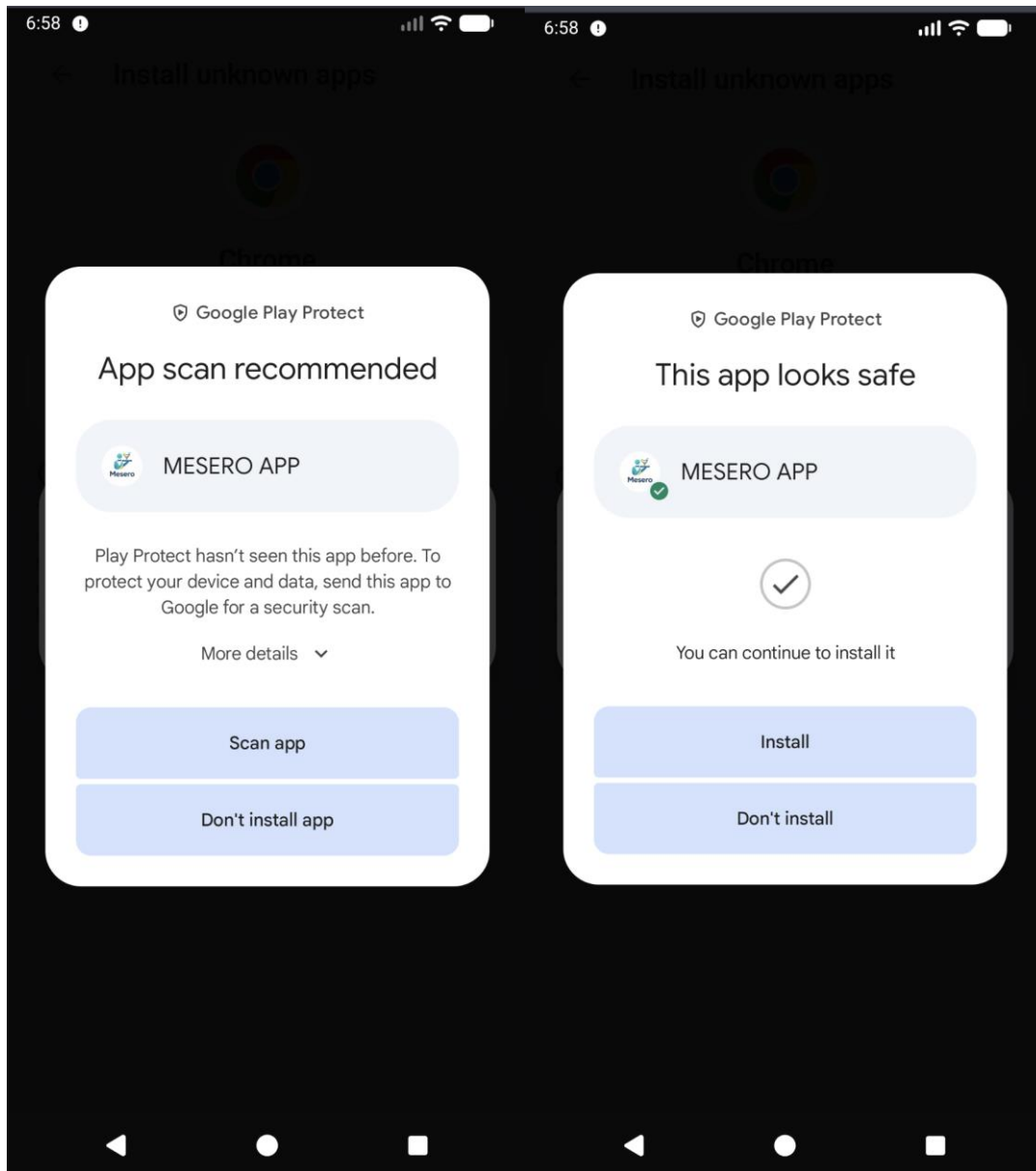
Para poder instalar el proyecto necesitaremos acceder al [github](https://github.com) y descargar la APK del proyecto, aquí saltara un aviso por la seguridad del dispositivo:

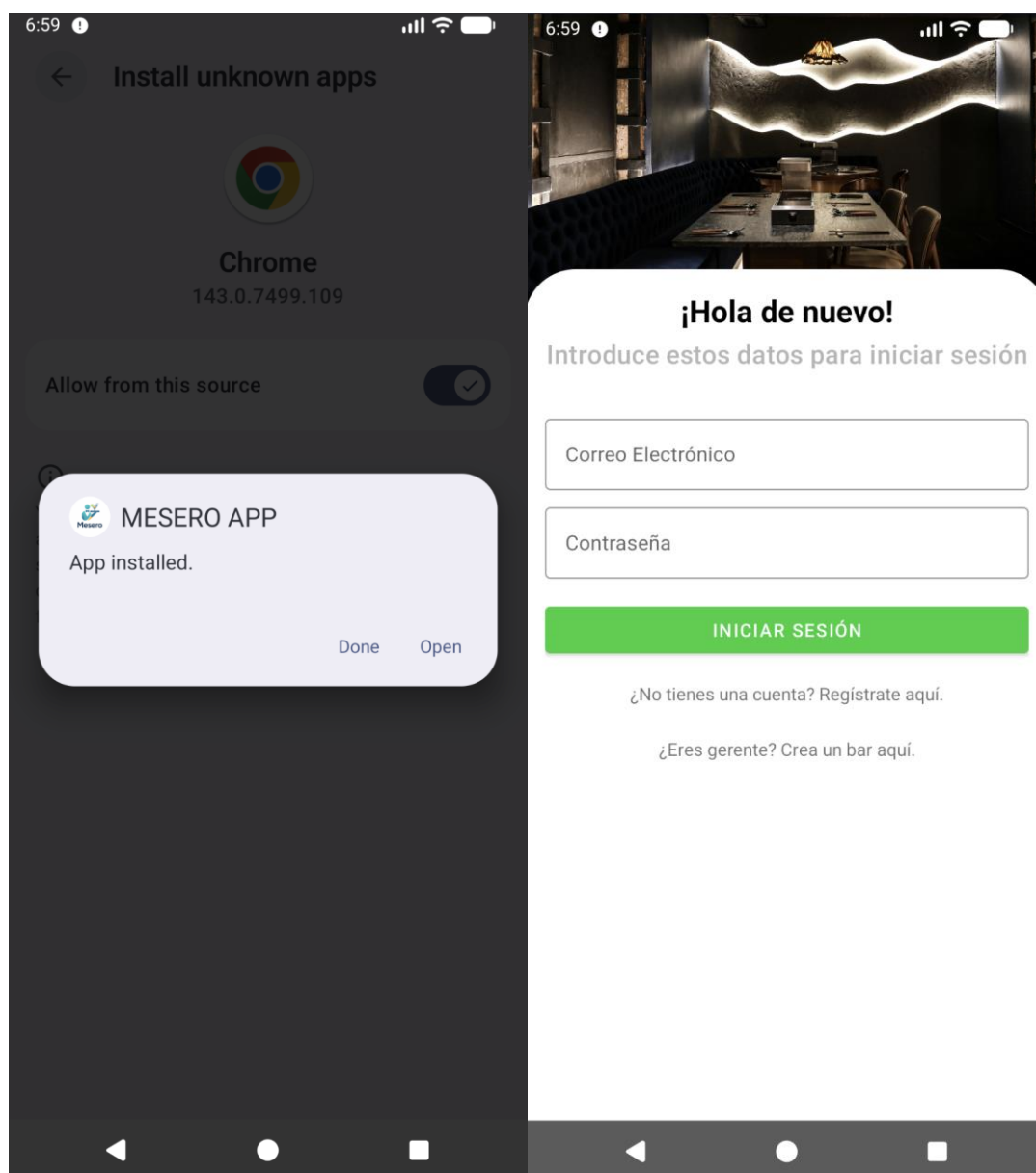


Tras ese aviso nos dira que por motivos de seguridad en el telefono debemos de permitir realizar instalaciones desde chrome. Seleccionamos settings, activamos el "Allow from this source" e instalamos



En este punto nos dirá que se debe de escanear la app al no ser instalada desde la playstore, aceptamos e instalamos

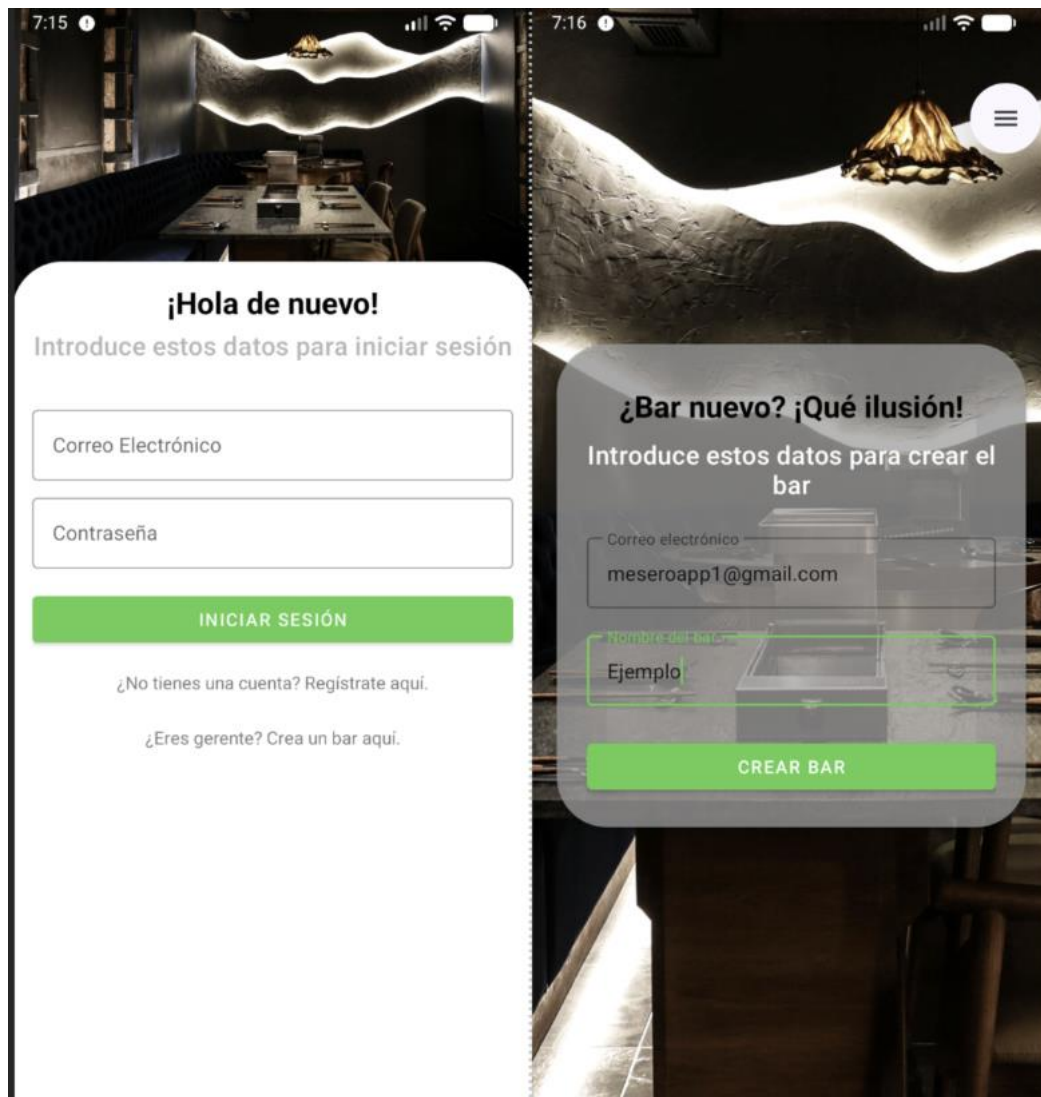




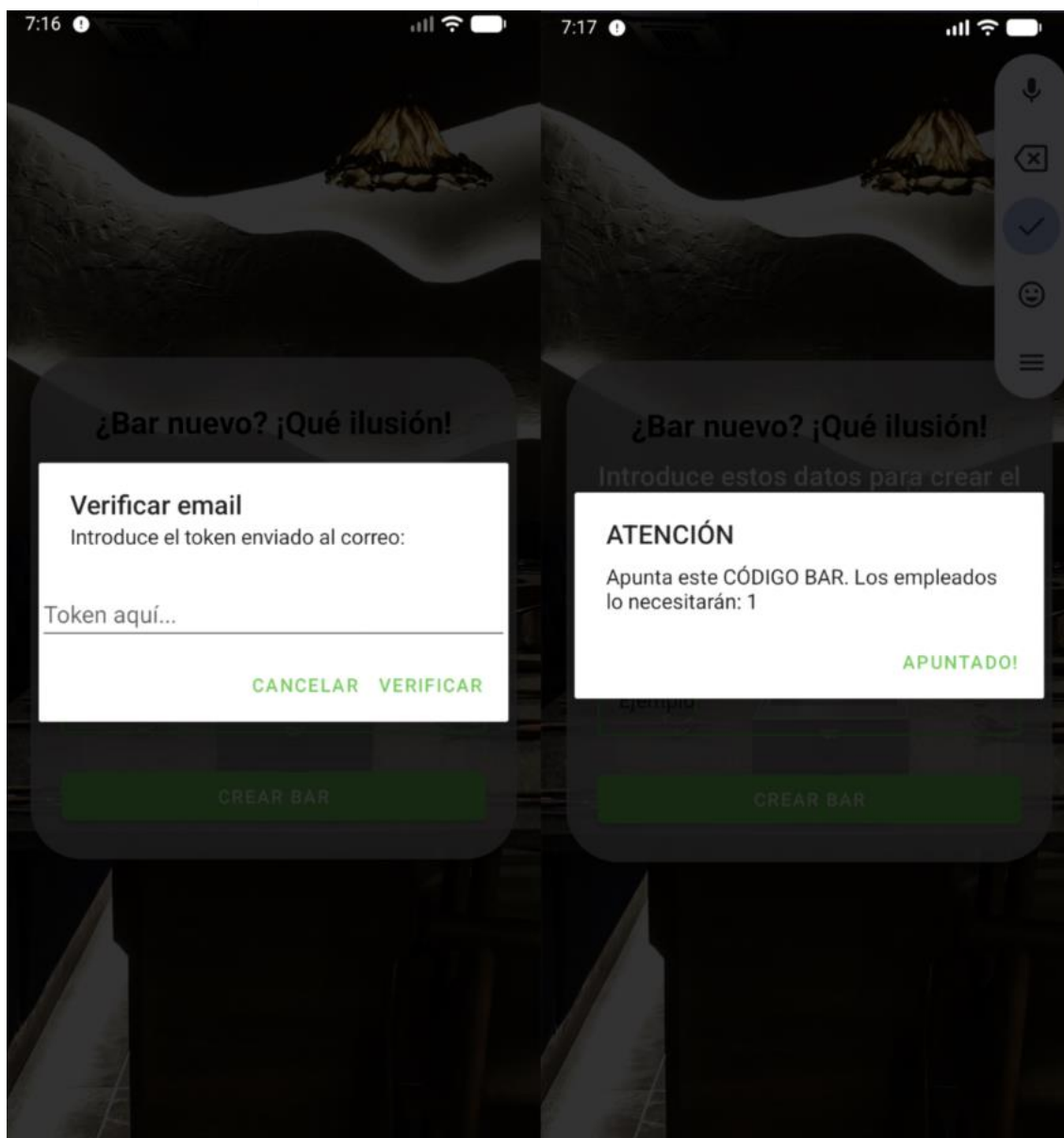
Manual de usuario:

Creación de bar y registro de usuarios

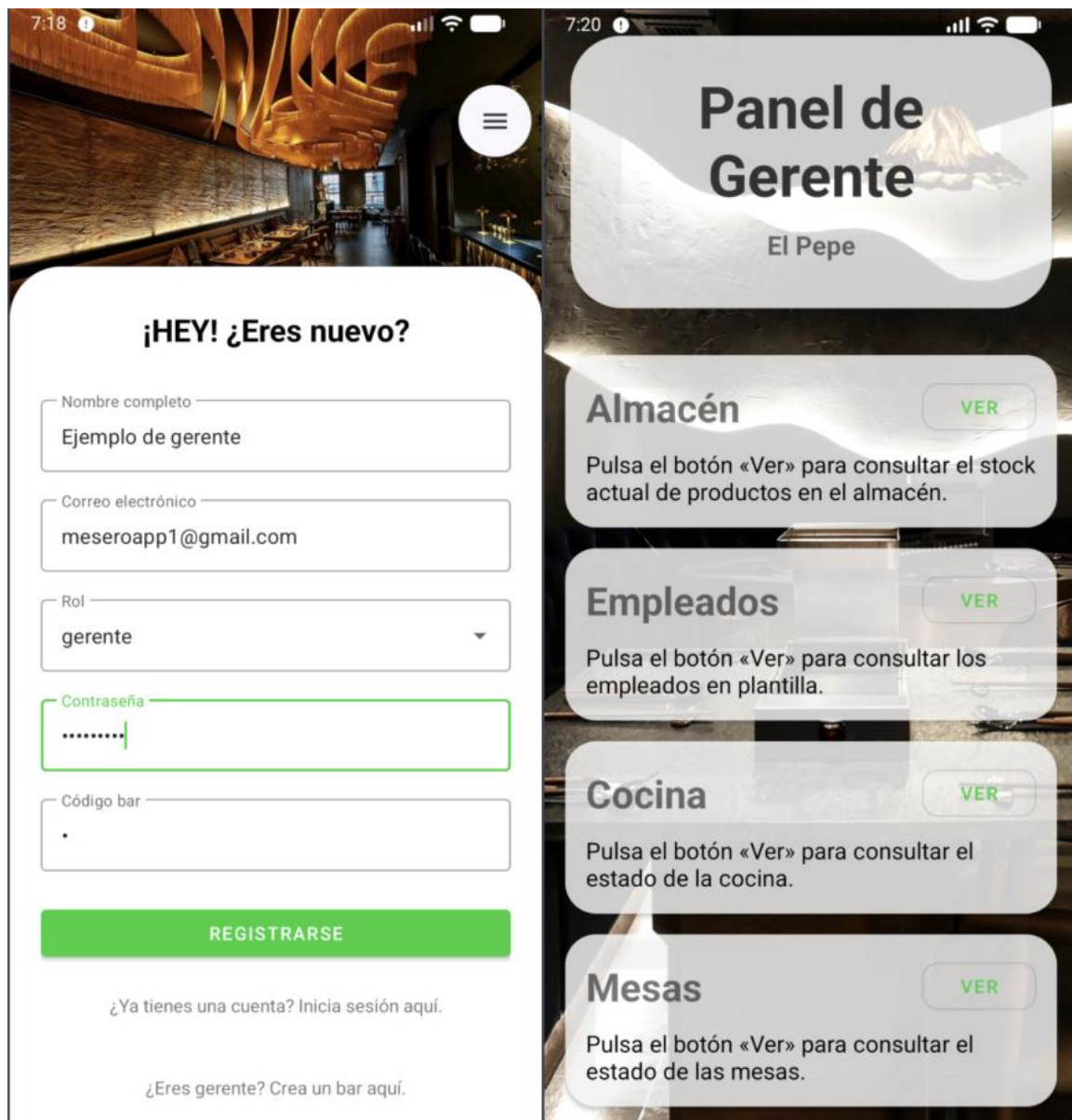
En la primera pantalla que aparece al instalar la app selecciona “Crear un bar aquí”, esta pantalla te dirigirá hasta otra pantalla en la que debes introducir el nombre del bar y el email.



Al hacer clic en “Crear bar” te aparece un pop up / modal en el que debes de introducir el código que te llegó al email indicado y se te proporcionara el código necesario para el registro de los empleados.



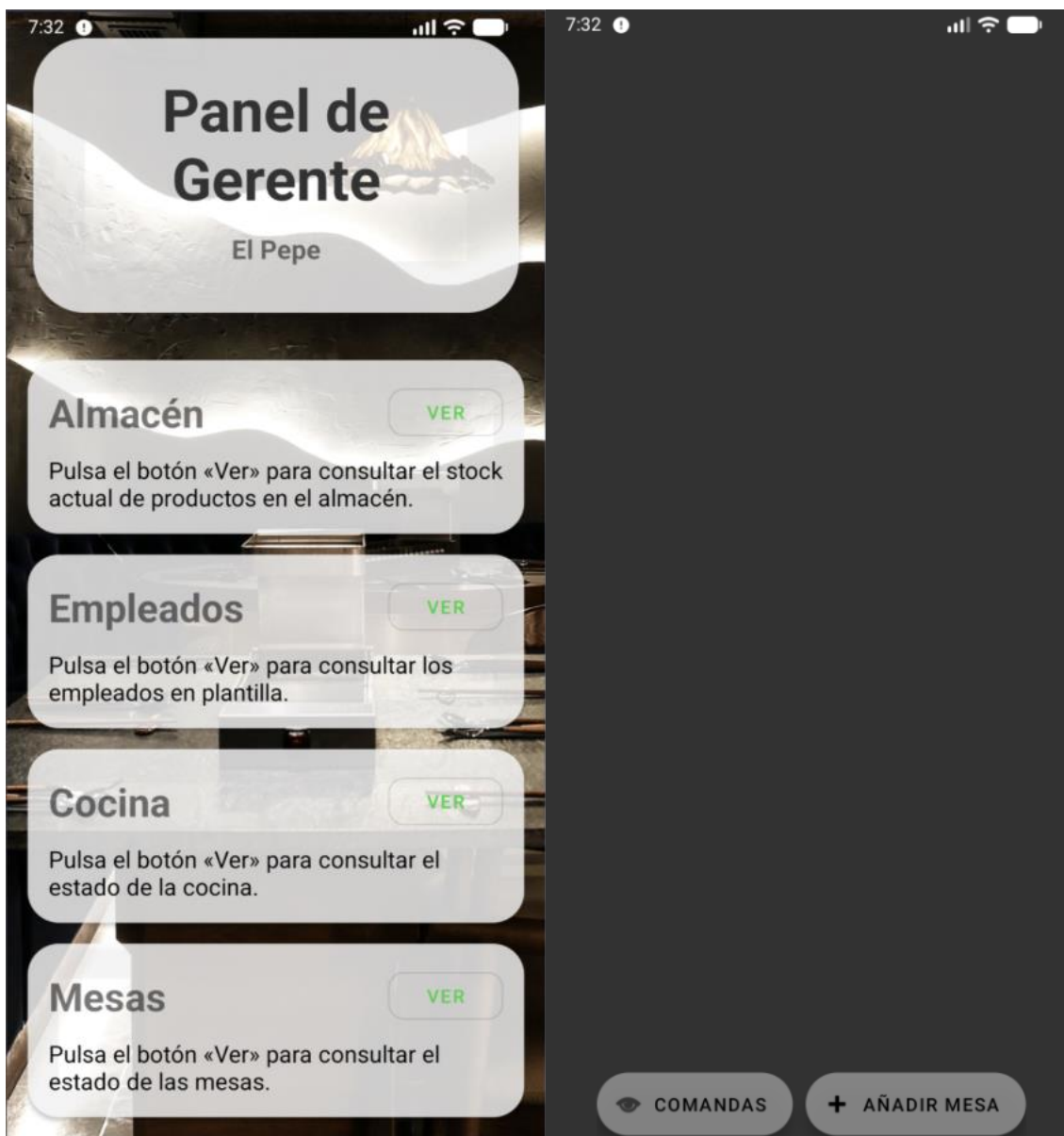
Una vez creado el bar deberás de registrarte como un nuevo usuario con el código anteriormente indicado y de nuevo se enviará un email al correo del bar indicando el código de verificación.



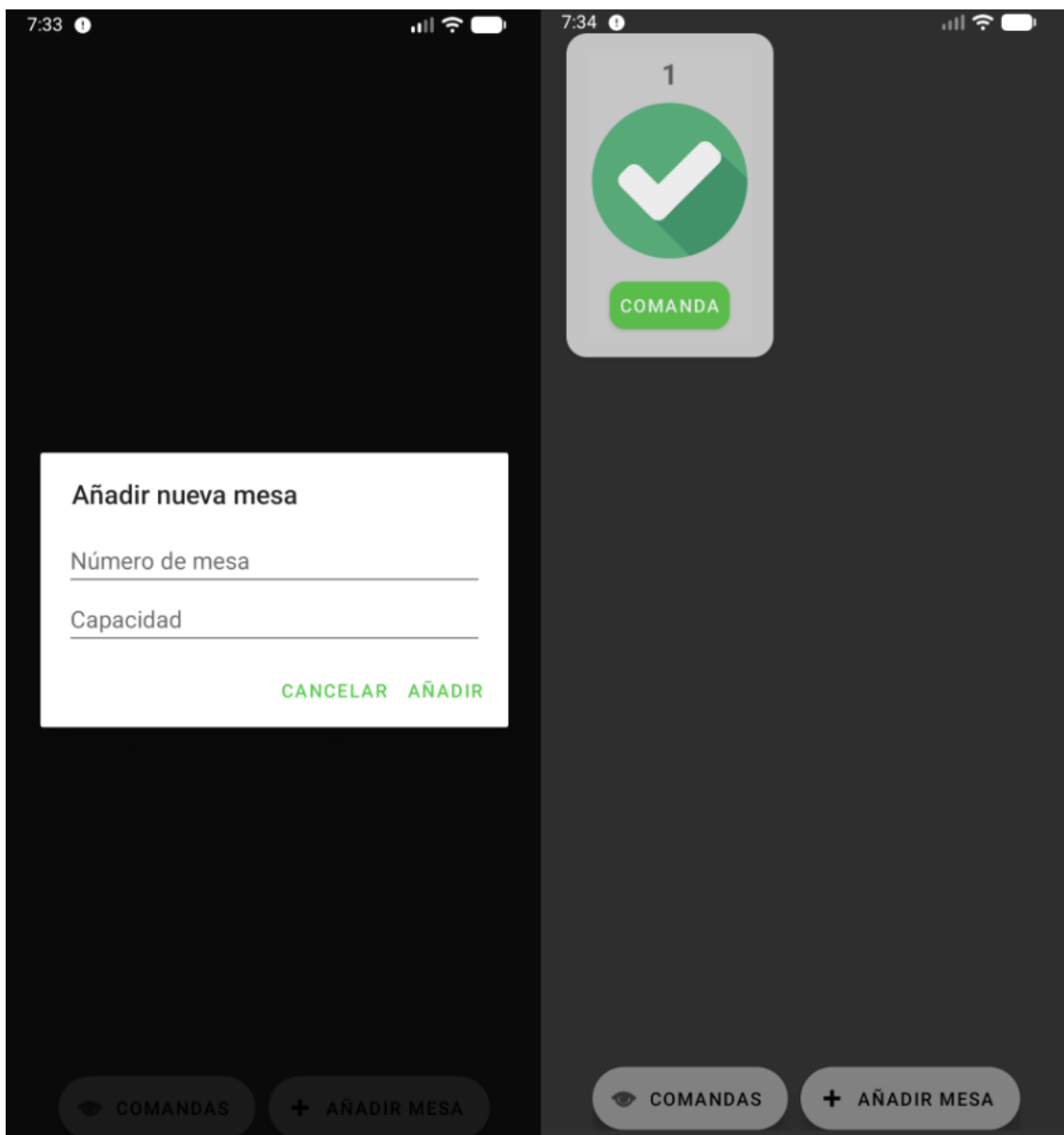
Aquí pide permitir notificaciones para enviar el sonido cuando llegue una nueva comanda y ya podrás ver el menú de gerente.

Añadir una mesa

Para añadir una mesa debes de acceder al menú de mesas con el botón ver y hacer clic en “+ AÑADIR MESA”

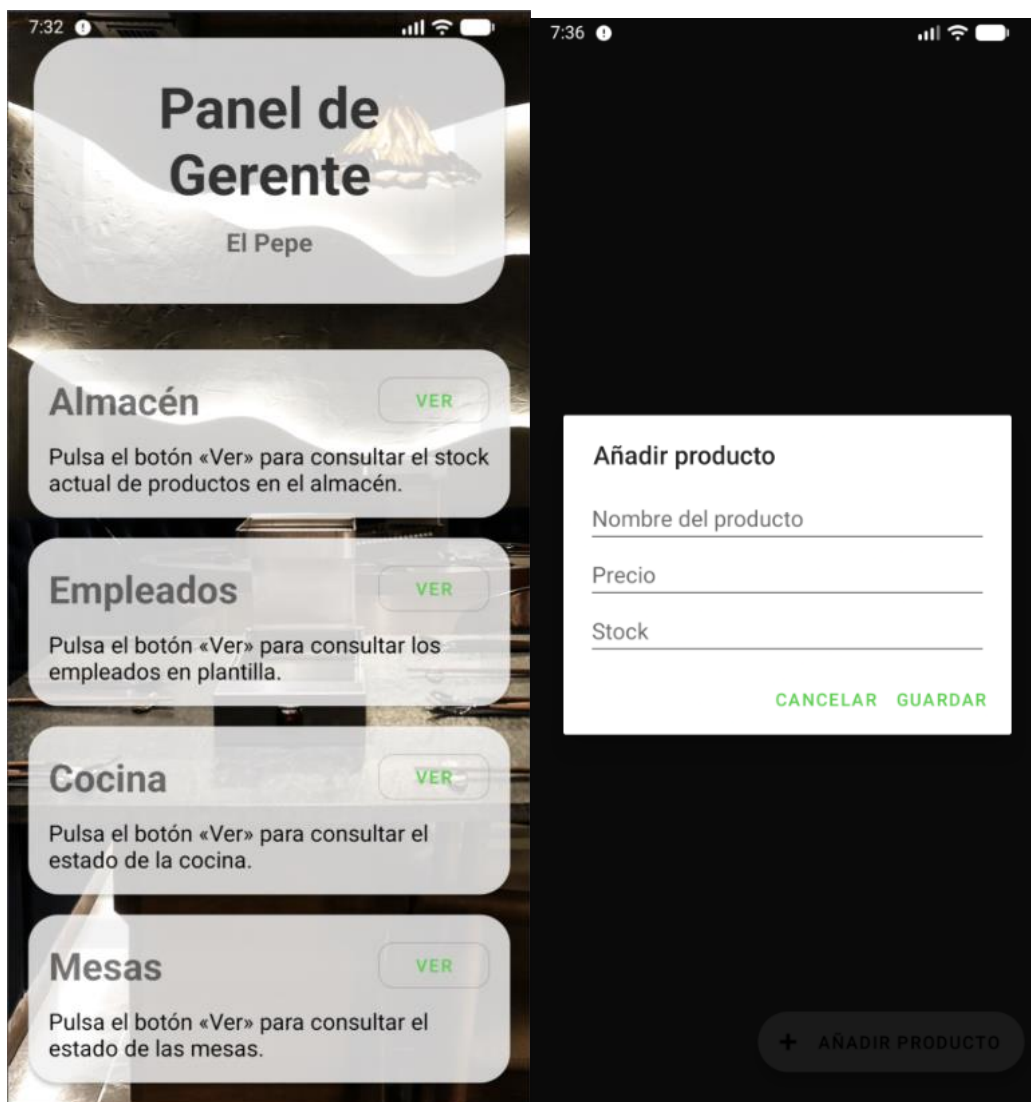


En esta te aparecerá un POP UP para añadir el número de mesa y la capacidad, una vez introducido ya aparecerá la mesa como disponible.

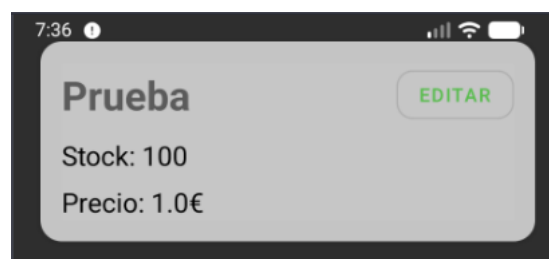


Añadir un producto

Para añadir un producto desde el panel de gerente debes de hacer clic en el botón ver del apartado almacén, haciendo clic en añadir producto te aparecerá un POP UP para introducir la información de este:

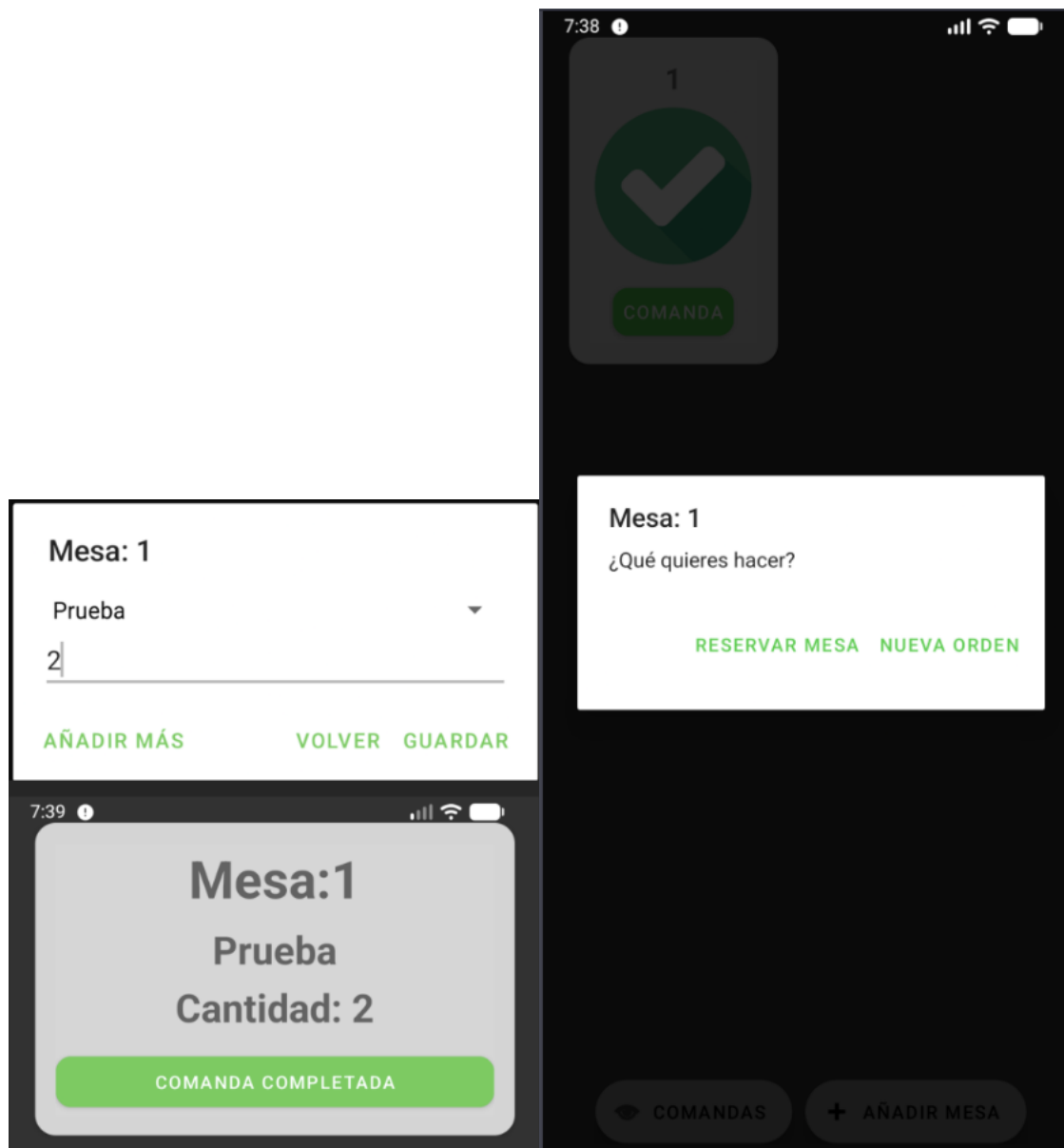


Aquí ya aparecerá el producto en el almacén:



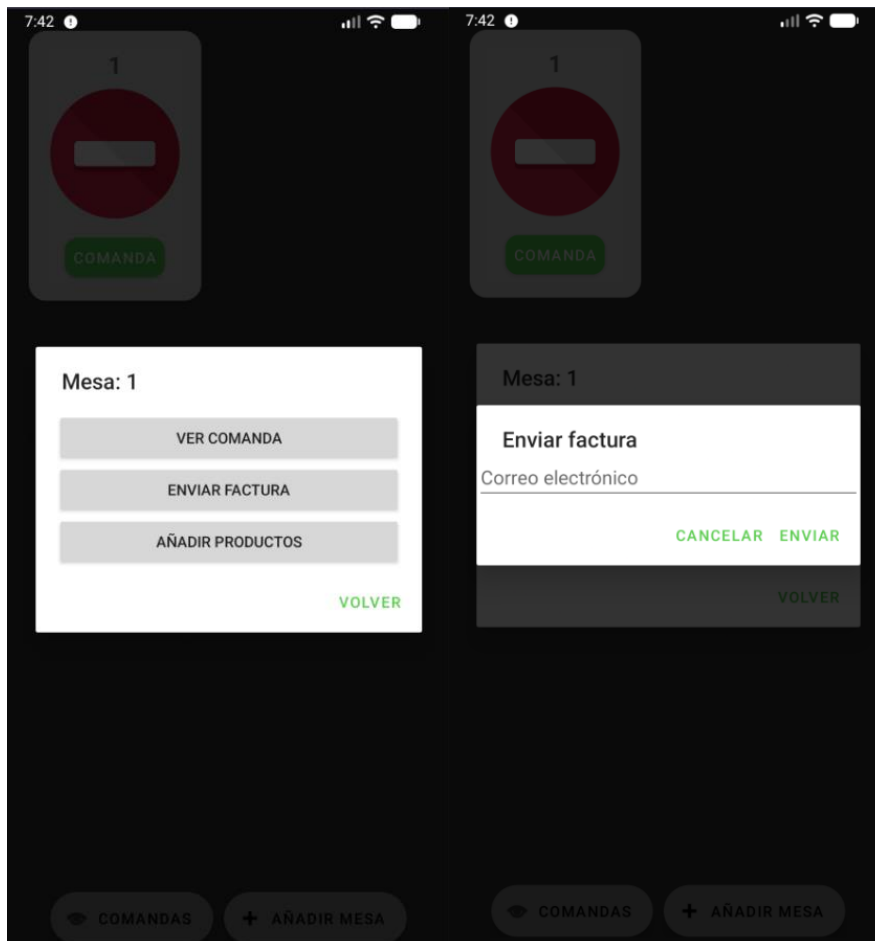
Crear una orden

En el apartado de mesas selección amos la mesa haciendo clic en comanda y seleccionamos nueva orden, aquí nos aparecerá la lista de productos y cantidad además si queremos añadir otros pulsamos añadir más y posteriormente guardar, así la comanda se envía a cocina.



Cerrar orden y enviar factura

En el apartado de mesas seleccionamos la mesa a la que queremos enviar la factura y aparecerá este pop up, seleccionamos enviar factura y añadimos el correo del cliente:



MeseroApp <meseroapp1@gmail.com> 7:43 PM (0 minutes ago) ☆ 😊 ↩ ⋮
to me ▾

Factura - Ejemplo

Producto	Cantidad	Precio	Subtotal
Prueba	2	1.0€	2.0€
Total			2.0€

Gracias por tu visita